

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

MediSync AI: AI-Powered Triage Chatbot, Prescription Digitizer, EHR & Pharmacy Portal

Architecture: Monorepo (pnpm + Turborepo) | **Environment:** Native / No-Docker **Version:** 2.0 **Date:** July 19, 2026

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) defines the structural, functional, and non-functional engineering blueprints for MediSync AI. This document establishes the strict specifications for the web and mobile application layout, relational schemas, AI routing workflows, and deployment frameworks. It incorporates a unified monorepo architecture using pnpm Workspaces and Turborepo to manage package dependencies, run parallel tasks, and share utilities across apps.

1.2 Scope

MediSync AI is an integrated ecosystem designed to resolve healthcare delivery bottlenecks. The system covers:

- * Real-time AI conversational symptom triage with critical risk assessment filters.
- * Computer-vision-based OCR and NLP parsing of handwritten and printed prescriptions.
- * Active chemical ingredient (salt composition) parsing and generic drug cross-mapping.
- * Real-time pharmacy inventory tracking and a patient-facing local medicine locator.
- * Centralized Electronic Health Records (EHR) secured via Time-based One-Time Password (TOTP) and Role-Based Access Control (RBAC).

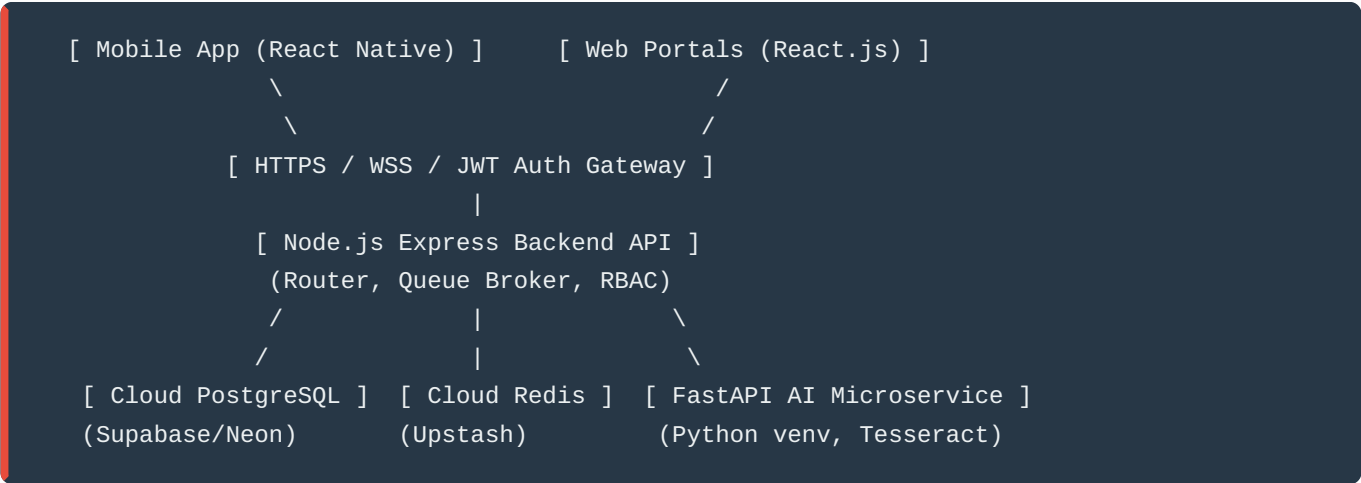
1.3 Stakeholders & User Roles

Role	Type	Key Responsibilities	System Access Level
Patient	Primary User	Submits symptom queries, uploads physical prescriptions, searches for local medicine availability, sets up adherence alerts, and generates OTP tokens for doctors.	Full read/write access to personal data partitions only.
Doctor	Professional User	Requests patient record access via OTP verification, updates active diagnosis fields, inputs clinical observations, and schedules follow-ups.	Full read access to verified patient records; append-only access to clinical session logs.
Pharmacy	Vendor / Partner	Manages local store inventory, updates stock levels and pricing for specific medicines, and makes their store discoverable to local patients.	Read/write access strictly to their specific pharmacy inventory partition.
System Admin	Operations Staff	Maintains master drug indexes, overrides system flags, updates active salt substitution indices, and handles emergency service definitions.	Full database access except unencrypted patient PII and raw authentication credentials.

2. SYSTEM OVERVIEW & MONOREPO ARCHITECTURE

2.1 High-Level Architecture

The application uses a decoupled, bare-metal local architecture utilizing a monorepo setup to ensure the mobile app and web portals share the exact same database interfaces and utility types.



2.2 Concrete Codebase Repository Directory Blueprint

The codebase is structured as a monorepo.

```
medisync-ai/
├─ package.json           # Root workspace configuration
├─ pnpm-workspace.yaml    # Workspace packages declaration
├─ turbo.json             # Turborepo execution pipeline configuration
├─ .env.example
├─ apps/
│   └─ backend/           # Node.js + Express API Gateway
│   └─ mobile-app/        # React Native Patient Application
│   └─ web-portals/       # React.js + Tailwind CSS Admin/Doctor/Pharmacy Portal
│   └─ ai-service/        # FastAPI Python Engine (Native execution)
├─ packages/
│   └─ shared-types/      # Common TypeScript interfaces & schemas
│       ├── package.json
│       └─ src/
│           └─ index.ts
└─ database/
    ├── init.sql
    └─ seed.sql
```

2.3 Shared Types Integration

To maintain structural symmetry between your API backend, patient mobile app, and doctor portal, the `@medisync/shared-types` module acts as a lightweight, shareable TypeScript package for system interfaces.

`packages/shared-types/src/index.ts`

```
export interface UserProfile {
  id: string;
  fullName: string;
  email: string;
  role: 'PATIENT' | 'DOCTOR' | 'PHARMACY';
}

export interface Prescription {
  id: string;
  patientId: string;
  doctorName: string;
  digitizedNotes: string;
  suggestedGenericAlternatives: string[];
  createdAt: string;
}

export interface StockCheckMessage {
  id: string;
  senderId: string;
  receiverId: string;
  medicineQuery: string;
  isAvailable: boolean;
  timestamp: string;
}
```

3. SYSTEM INSTALLATION & NATIVE EXECUTION BLUEPRINT

To run the application natively (without Docker), the environment must be configured at the host OS level.

3.1 OS-Level Prerequisites

- **Node.js (v18+) & pnpm (v9.0.0+).**
- **Python (v3.11+):** Required for the AI FastAPI service.
- **Tesseract OCR:** Must be installed on the host OS (`brew install tesseract` on Mac, or via `.exe` on Windows).
- **Databases:** Obtain connection strings for a hosted PostgreSQL (e.g., Supabase) and a hosted Redis (e.g., Upstash).

3.2 Workspace & Turborepo Configuration

The root `package.json` configures workspace dependency management and maps global scripts to build and run all sub-projects. The `pnpm-workspace.yaml` registers the `apps/*` and `packages/*` folders as active packages within the workspace pipeline. Turborepo acts as the execution engine for the workspaces.

3.3 Step-by-Step Native Execution Sequence

Because services are not containerized, you will run the AI service manually, while Turborepo handles the rest.

Terminal 1: AI Service (Python)

```
cd apps/ai-service
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn main:app --host 127.0.0.1 --port 8000 --reload
```

Terminal 2: Monorepo Orchestration (Node.js) 1. Run `pnpm install` in your root directory to recursively link and optimize packages across all apps. 2. Inject shared types by adding `"@medisync/shared-types": "workspace:*"` to local `package.json` dependencies. 3. Run `pnpm dev` in the root folder. 4. Turborepo will start the backend server on its local port, the doctor web portal dashboard, and the React Native Metro Bundler for your mobile app simultaneously.

4. FUNCTIONAL REQUIREMENTS MODULES

4.1 Pharmacy & Inventory Locator Module (PL)

This module connects patients looking for prescribed drugs with local pharmacies that have verified stock.

ID	Feature	Description	Priority	Actor(s)	Input	Output / Result
PL-001	Inventory Management	Allows pharmacists to update their in-store catalog, marking items as IN STOCK, OUT OF STOCK, and setting current shelf prices.	Must Have	Pharmacy	Drug IDs, stock status booleans, price floats.	Database updates to <code>pharmacy_inventory</code> table.
PL-002	Medicine Locator	Patients can query an extracted prescription item to see a map/list of verified pharmacies nearby holding that exact brand.	Must Have	Patient	Active drug ID or salt string.	A tabular list of pharmacies with addresses, contact info, and live stock status.

4.2 AI-OCR & Prescription Digitization Module (OD)

ID	Feature	Description	Priority	Actor(s)	Input	Output / Result
OD-001	Optical Ingestion	Receives image data captured directly from client cameras.	Must Have	Patient	Multi-part image streams.	Structured storage file locations.
OD-002	NLP Extraction	Processes raw text blobs using structured parsing to capture branded titles and dosages.	Must Have	System	Raw extracted character text blocks.	Structured JSON array listing separated medicine records.

5. CORE ENGINE ALGORITHMS & CRITICAL DATA SCHEMAS

5.1 Generic Medicine Matching Optimization Logic

To calculate alternative matching brand entities based on the patient's budget, the scoring matrix uses the following optimization approach:

$$\text{Score} = (0.60 \times P_{\text{score}}) + (0.40 \times Q_{\text{score}})$$

Where: * $P_{\text{score}} = \max(0, (\text{Branded Price} - \text{Generic Alternative Price}) / \text{Branded Price}) \times 100$ * $Q_{\text{score}} = \text{System Trust Rating (1.0 to 5.0)} \times 20$

5.2 Updated PostgreSQL DDL Schema Script

The database schema includes **PHARMACY** roles and inventory mapping tables to support the cross-platform applications.

```

CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE TYPE user_role_enum AS ENUM ('PATIENT', 'DOCTOR', 'ADMIN', 'PHARMACY');
CREATE TYPE booking_status_enum AS ENUM ('PENDING', 'CONFIRMED', 'CANCELLED',
'COMPLETED');
CREATE TYPE alert_status_enum AS ENUM ('ACTIVE', 'SUSPENDED', 'ARCHIVED');

CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role user_role_enum NOT NULL DEFAULT 'PATIENT',
    full_name VARCHAR(255) NOT NULL,
    phone_number VARCHAR(50),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE master_pharmacy (
    drug_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    brand_name VARCHAR(255) NOT NULL,
    salt_composition VARCHAR(255) NOT NULL,
    strength VARCHAR(50) NOT NULL,
    estimated_price NUMERIC(10, 2) NOT NULL,
    is_generic BOOLEAN DEFAULT false,
    trust_rating NUMERIC(3, 2) DEFAULT 5.00
);

CREATE TABLE pharmacies (
    pharmacy_id UUID PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
    store_name VARCHAR(255) NOT NULL,
    license_number VARCHAR(100) UNIQUE NOT NULL,
    street_address TEXT NOT NULL,
    city VARCHAR(100) NOT NULL,
    is_verified BOOLEAN DEFAULT false
);

CREATE TABLE pharmacy_inventory (
    inventory_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    pharmacy_id UUID NOT NULL REFERENCES pharmacies(pharmacy_id) ON DELETE CASCADE,
    drug_id UUID NOT NULL REFERENCES master_pharmacy(drug_id) ON DELETE CASCADE,
    in_stock BOOLEAN DEFAULT true,
    current_price NUMERIC(10, 2),
    last_updated TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(pharmacy_id, drug_id)
);

```


6. CORE ENDPOINTS, INTERFACES & SYSTEM RISK COMPLIANCE

6.1 Comprehensive API Endpoints Specification Matrix

Verb	Path	Request Headers / Guard Parameter	JSON Payload Schema Payload	Target Functional Response Pattern
POST	/api/auth/register	Open Public Endpoint Access	<pre>{"email": "...", "role": "PATIENT"}</pre>	211 Created → <pre>{"token": "JWT_BLOB"}</pre>
POST	/api/digitize/ocr	Bearer Token (Patient Access)	Multi-part Form Data (file binary)	200 OK → <pre>{"medicines": [...]}</pre>
GET	/api/drugs/substitutes	Bearer Token (Patient Access)	Query: ? saltComposition=Paracetamol	200 OK → <pre>[{"brandName": "X", "price": 12.50}]</pre>
POST	/api/inventory/update	Bearer Token (Pharmacy)	<pre>{"drugId": "UUID", "inStock": true, "price": 15.00}</pre>	200 OK → <pre>{"status": "Inventory Updated"}</pre>
GET	/api/inventory/locate	Bearer Token (Patient Access)	Query: ?drugId=UUID&city=Dhaka	200 OK → <pre>[{"store_name": "MediCare", "in_stock": true}]</pre>

6.2 Data Security Enforcement Matrix

- **Data In-Transit Enforcements:** All network interactions across the monorepo must force explicit TLS 1.3 protections.
- **Data At-Rest Enforcements:** Patient diagnosis rows and prescription contents must be encrypted before being written to disk using AES-256 block structures.